API REST Con Laravel 11+ (Versión Modernizada)

Contenidos

1 API REST con Laravel 11+ (Versión Modernizada)

1.1 Introducción a los cambios en Laravel 11+

Desde Laravel 10, ha habido cambios significativos en cómo se estructura y configura una API REST:

1.1.1 Cambios principales desde Laravel 10:

1. **Sanctum es ahora la opción preferida** para autenticación de API en lugar de JWT
2. **Estructura de rutas mejorada** con mejor separación de concerns
3. **Middleware más flexible** y fácil de usar
4. **Configuración de CORS simplificada**
5. **Validación de Request mejorada**
6. **Mejor soporte para API versioning**

1.2 ¿Qué es Laravel Sanctum?

Laravel Sanctum es un paquete oficial de Laravel que proporciona autenticación simple basada en tokens para aplicaciones SPA (Single Page Applications) y APIs móviles.

1.2.1 Ventajas de Sanctum sobre JWT:

- Más simple de configurar
- Soporta CSRF protection
- Gestión automática de tokens
- Mejor seguridad por defecto
- Integración nativa con Laravel
- No requiere configuración adicional de secretos

1.3 Creación de una Aplicación API REST con Laravel 11+

1.3.1 Requisitos previos:

- PHP >= 8.2
- Composer

- Docker (para usar Sail, opcional)
- MySQL o SQLite

1.3.2 Instalación con Sail

Para crear una nueva aplicación Laravel 11 con Sail:

```
curl -s "https://laravel.build/api-rest-app?with=mysql,redis" | bash
```

O si prefieres especificar la versión explícitamente:

```
composer create-project laravel/laravel api-rest-app --prefer-dist
cd api-rest-app
./vendor/bin/sail up
```

1.3.3 Iniciar aplicación existente:

Si ya tienes la aplicación, solo necesitas:

```
sail up -d
sail artisan migrate
```

1.4 Configuración de Laravel Sanctum

1.4.1 Instalación

Sanctum viene preinstalado en Laravel 11+, pero si necesitas instalarlo manualmente:

```
sail composer require laravel/sanctum
```

1.4.2 Publicar configuración

```
sail artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
```

Esto crea el archivo `config/sanctum.php`

1.4.3 Ejecutar migraciones

```
sail artisan migrate
```

Esto crea la tabla `personal_access_tokens` necesaria para almacenar los tokens.

1.4.4 Configuración de CORS

Edita `config/cors.php` para permitir que tu frontend consuma la API:

```php
<?php

return [
    'paths' => ['api/*', 'sanctum/csrf-cookie'],
    'allowed_methods' => ['*'],
    'allowed_origins' => [
        'http://localhost:3000',      // Vue/React dev
        'http://localhost:5173',      // Vite dev
        'http://localhost:8000',      // Otro proyecto
    ],
    'allowed_origins_patterns' => [],
```

```
    'allowed_headers' => ['*'],
    'exposed_headers' => ['Authorization'],
    'max_age' => 0,
    'supports_credentials' => true,
];
```

1.4.5 Middleware de Sanctum en rutas API

En el archivo `routes/api.php`, las rutas protegidas deben usar el middleware `auth:sanctum`:

```
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
    return $request->user();
});
```

1.5 Modelo, Migraciones y Controlador

1.5.1 Crear el modelo Post con migraciones

```
sail artisan make:model Post -m
```

1.5.2 Editar la migración

Abre `database/migrations/YYYY_MM_DD_HHMMSS_create_posts_table.php` y actualiza el método up:

```
<?php

public function up(): void
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->cascadeOnDelete();
        $table->string('title', 255);
        $table->string('image', 255)->nullable();
        $table->mediumText('description');
        $table->boolean('hidden')->default(false);
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('posts');
}
```

1.5.3 Configurar el modelo Post

Edita `app/Models/Post.php`:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Post extends Model
{
    use HasFactory;

    protected $fillable = [
        'title',
        'description',
        'image',
        'hidden',
    ];

    protected $casts = [
        'created_at' => 'datetime',
        'updated_at' => 'datetime',
        'hidden' => 'boolean',
    ];

    /**
     * Get the user that owns the post.
     */
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

1.5.4 Configurar el modelo User

Edita `app/Models/User.php` para permitir crear tokens:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
```

```
    protected $fillable = [
        'name',
        'email',
        'password',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected $casts = [
        'email_verified_at' => 'datetime',
        'password' => 'hashed',
    ];

    /**
     * Get the posts for the user.
     */
    public function posts()
    {
        return $this->hasMany(Post::class);
    }
}
```

1.5.5 Crear el controlador de autenticación

```
sail artisan make:controller Api/AuthController
```

Edita app/Http/Controllers/Api/AuthController.php:

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules\Password;
use Illuminate\Http\Response;

class AuthController extends Controller
{
    /**
     * Register a new user.
     */
    public function register(Request $request): Response
    {
        $validated = $request->validate([
```

```
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|max:255|unique:users',
            'password' => ['required', 'confirmed', Password::defaults()],
        ]);

        $user = User::create([
            'name' => $validated['name'],
            'email' => $validated['email'],
            'password' => Hash::make($validated['password']),
        ]);

        $token = $user->createToken('auth_token')->plainTextToken;

        return response([
            'user' => $user,
            'token' => $token,
        ], Response::HTTP_CREATED);
    }

    /**
     * Login user and create token.
     */
    public function login(Request $request): Response
    {
        $validated = $request->validate([
            'email' => 'required|string|email',
            'password' => 'required|string',
        ]);

        $user = User::where('email', $validated['email'])->first();

        if (!$user || !Hash::check($validated['password'], $user->password)) {
            return response([
                'message' => 'Invalid credentials',
            ], Response::HTTP_UNAUTHORIZED);
        }

        $token = $user->createToken('auth_token')->plainTextToken;

        return response([
            'user' => $user,
            'token' => $token,
        ], Response::HTTP_OK);
    }

    /**
     * Get authenticated user.
     */
    public function me(Request $request): Response
```

```
    {
        return response([
            'user' => $request->user(),
        ], Response::HTTP_OK);
    }

    /**
     * Logout user (revoke token).
     */
    public function logout(Request $request): Response
    {
        $request->user()->currentAccessToken()->delete();

        return response([
            'message' => 'Successfully logged out',
        ], Response::HTTP_OK);
    }
}
```

1.5.6 Crear el controlador de Posts

```
sail artisan make:controller Api/PostController --resource
```

Edita app/Http/Controllers/Api/PostController.php:

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Resources\PostResource;
use App\Models\Post;
use Illuminate\Http\Request;
use Illuminate\Http\Response;
use Illuminate\Support\Facades\Storage;

class PostController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index(): Response
    {
        $posts = Post::where('hidden', false)
            ->latest()
            ->paginate(15);

        return response([
            'data' => PostResource::collection($posts),
            'meta' => [
                'total' => $posts->total(),
```

```
                'per_page' => $posts->perPage(),
                'current_page' => $posts->currentPage(),
                'last_page' => $posts->lastPage(),
            ],
        ], Response::HTTP_OK);
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request): Response
    {
        $validated = $request->validate([
            'title' => 'required|string|max:255',
            'description' => 'required|string|max:2000',
            'image' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:5120',
        ]);

        $post = new Post($validated);
        $post->user_id = auth()->id();

        if ($request->hasFile('image')) {
            $path = $request->file('image')->store('posts', 'public');
            $post->image = $path;
        }

        $post->save();

        return response([
            'data' => new PostResource($post),
            'message' => 'Post created successfully',
        ], Response::HTTP_CREATED);
    }

    /**
     * Display the specified resource.
     */
    public function show(Post $post): Response
    {
        if ($post->hidden) {
            return response([
                'message' => 'Post not found',
            ], Response::HTTP_NOT_FOUND);
        }

        return response([
            'data' => new PostResource($post),
        ], Response::HTTP_OK);
    }
```

```php
    /**
     * Update the specified resource in storage.
     */
    public function update(Request $request, Post $post): Response
    {
        // Verificar autorización
        if (auth()->id() !== $post->user_id) {
            return response([
                'message' => 'Unauthorized to update this post',
            ], Response::HTTP_FORBIDDEN);
        }

        $validated = $request->validate([
            'title' => 'sometimes|string|max:255',
            'description' => 'sometimes|string|max:2000',
            'image' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:5120',
        ]);

        if (isset($validated['title'])) {
            $post->title = $validated['title'];
        }

        if (isset($validated['description'])) {
            $post->description = $validated['description'];
        }

        if ($request->hasFile('image')) {
            // Eliminar imagen anterior si existe
            if ($post->image && Storage::disk('public')->exists($post->image)) {
                Storage::disk('public')->delete($post->image);
            }

            $path = $request->file('image')->store('posts', 'public');
            $post->image = $path;
        }

        $post->save();

        return response([
            'data' => new PostResource($post),
            'message' => 'Post updated successfully',
        ], Response::HTTP_OK);
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(Post $post): Response
```

```
    {
        // Verificar autorización
        if (auth()->id() !== $post->user_id) {
            return response([
                'message' => 'Unauthorized to delete this post',
            ], Response::HTTP_FORBIDDEN);
        }

        // Eliminar imagen si existe
        if ($post->image && Storage::disk('public')->exists($post->image)) {
            Storage::disk('public')->delete($post->image);
        }

        $post->delete();

        return response([
            'message' => 'Post deleted successfully',
        ], Response::HTTP_OK);
    }
}
```

1.6 Resources para formatear respuestas

1.6.1 Crear el Resource

```
sail artisan make:resource PostResource
```

Edita app/Http/Resources/PostResource.php:

```
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class PostResource extends JsonResource
{
    /**
     * Transform the resource into an array.
     *
     * @return array<string, mixed>
     */
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'title' => $this->title,
            'description' => $this->description,
            'image' => $this->image ? asset('storage/' . $this->image) : null,
            'author' => [
                'id' => $this->user->id,
```

```
                'name' => $this->user->name,
                'email' => $this->user->email,
            ],
            'created_at' => $this->created_at->toIso8601String(),
            'updated_at' => $this->updated_at->toIso8601String(),
        ];
    }
}
```

1.7 Rutas API

Edita `routes/api.php`:

```
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\PostController;

/*
|--------------------------------------------------------------------------
| API Routes
|--------------------------------------------------------------------------
|
| Here is where you can register API routes for your application. These
| routes are loaded by the RouteServiceProvider and all of them will
| be assigned to the "api" middleware group. Make something great!
|
*/

Route::prefix('v1')->group(function () {
    // Rutas públicas de autenticación
    Route::post('/auth/register', [AuthController::class, 'register']);
    Route::post('/auth/login', [AuthController::class, 'login']);

    // Rutas públicas de Posts (solo lectura)
    Route::get('/posts', [PostController::class, 'index']);
    Route::get('/posts/{post}', [PostController::class, 'show']);

    // Rutas protegidas (requieren autenticación)
    Route::middleware('auth:sanctum')->group(function () {
        Route::post('/auth/logout', [AuthController::class, 'logout']);
        Route::get('/auth/me', [AuthController::class, 'me']);

        // Operaciones de escritura en posts
        Route::post('/posts', [PostController::class, 'store']);
        Route::put('/posts/{post}', [PostController::class, 'update']);
        Route::delete('/posts/{post}', [PostController::class, 'destroy']);
    });
```

```
});

// Ruta de prueba sin autenticación
Route::get('/health', function (Request $request) {
    return response()->json([
        'status' => 'OK',
        'message' => 'API is running',
        'timestamp' => now(),
    ]);
});
```

1.8 Crear usuarios de prueba

Edita `database/seeders/DatabaseSeeder.php`:

```
<?php

namespace Database\Seeders;

use App\Models\Post;
use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // Crear usuario de prueba
        $user = User::create([
            'name' => 'Test User',
            'email' => 'test@example.com',
            'password' => Hash::make('password'),
        ]);

        // Crear más usuarios
        User::factory(5)->create();

        // Crear posts para el usuario de prueba
        Post::factory(10)->for($user)->create();

        // Crear posts para otros usuarios
        User::all()->each(function ($user) {
            Post::factory(3)->for($user)->create();
        });
    }
}
```

Ejecuta el seeder:

```
sail artisan migrate:fresh --seed
```

1.9 Manejo de errores

1.9.1 Crear Exception Handler personalizado

Laravel 11+ tiene mejor manejo de excepciones. Edita `app/Exceptions/Handler.php`:

```php
<?php

namespace App\Exceptions;

use Illuminate\Foundation\Exceptions\Handler as ExceptionHandler;
use Illuminate\Http\Response;
use Illuminate\Database\Eloquent\ModelNotFoundException;
use Illuminate\Validation\ValidationException;
use Throwable;

class Handler extends ExceptionHandler
{
    /**
     * The list of the inputs that are never flashed to the session on validation exce
     *
     * @var array<int, string>
     */
    protected $dontFlash = [
        'current_password',
        'password',
        'password_confirmation',
    ];

    /**
     * Register the exception handling callbacks for the application.
     */
    public function register(): void
    {
        $this->reportable(function (Throwable $e) {
            //
        });

        $this->renderable(function (ModelNotFoundException $e, $request) {
            if ($request->is('api/*')) {
                return response()->json([
                    'message' => 'Resource not found',
                    'error' => 'NOT_FOUND',
                ], Response::HTTP_NOT_FOUND);
            }
        });

        $this->renderable(function (ValidationException $e, $request) {
```

```
            if ($request->is('api/*')) {
                return response()->json([
                    'message' => 'Validation failed',
                    'errors' => $e->errors(),
                ], Response::HTTP_UNPROCESSABLE_ENTITY);
            }
        });
    }
}
```

1.10 Probar la API

1.10.1 Obtener token de acceso

Con cURL:

```
curl -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "test@example.com",
    "password": "password"
  }'
```

Respuesta:

```
{
    "user": {
        "id": 1,
        "name": "Test User",
        "email": "test@example.com",
        "email_verified_at": null,
        "created_at": "2024-01-15T10:30:00.000000Z",
        "updated_at": "2024-01-15T10:30:00.000000Z"
    },
    "token": "1|abc123def456ghi789jkl012mno345pqr678stu"
}
```

1.10.2 Crear un post (requiere autenticación)

```
curl -X POST http://localhost:8000/api/v1/posts \
  -H "Authorization: Bearer 1|abc123def456ghi789jkl012mno345pqr678stu" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Mi primer post",
    "description": "Esta es una descripción interesante"
  }'
```

1.10.3 Listar posts (público)

```
curl http://localhost:8000/api/v1/posts
```

1.10.4 Obtener un post específico

```
curl http://localhost:8000/api/v1/posts/1
```

1.10.5 Actualizar un post

```
curl -X PUT http://localhost:8000/api/v1/posts/1 \
  -H "Authorization: Bearer 1|abc123def456ghi789jkl012mno345pqr678stu" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Título actualizado",
    "description": "Nueva descripción"
  }'
```

1.10.6 Eliminar un post

#+begin_src bash curl -X DELETE http://localhost:8000/api/v1/posts/1 \ -H "Authorization: Bearer 1|abc123def456ghi789jkl012mno345pqr678stu" #+end_str>

1.11 Validación mejorada con Form Requests

Crea un Form Request personalizado:

```
sail artisan make:request StorePostRequest
sail artisan make:request UpdatePostRequest
```

Edita app/Http/Requests/StorePostRequest.php:

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StorePostRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return auth()->check();
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array|str
     */
    public function rules(): array
    {
        return [
            'title' => 'required|string|max:255',
            'description' => 'required|string|max:2000',
            'image' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:5120',
        ];
    }
```

```
    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'title.required' => 'El título es obligatorio.',
            'title.max' => 'El título no puede superar 255 caracteres.',
            'description.required' => 'La descripción es obligatoria.',
            'description.max' => 'La descripción no puede superar 2000 caracteres.',
            'image.image' => 'El archivo debe ser una imagen.',
            'image.max' => 'La imagen no puede superar 5MB.',
        ];
    }
}
```

Luego usa estos Form Requests en tu controlador:

#+begin_src php public function store(StorePostRequest $request): Response { $validated = $request->validated(); // ... resto del código } #+end_str>

1.12 Testing de la API

Crea tests para tus endpoints:

```
sail artisan make:test Feature/AuthTest
sail artisan make:test Feature/PostTest
```

Ejemplo en `tests/Feature/PostTest.php`:

```
<?php

namespace Tests\Feature;

use App\Models\Post;
use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class PostTest extends TestCase
{
    use RefreshDatabase;

    protected User $user;

    protected function setUp(): void
    {
        parent::setUp();
        $this->user = User::factory()->create();
    }
```

```
    public function test_can_list_posts(): void
    {
        Post::factory(5)->create();

        $response = $this->getJson('/api/v1/posts');

        $response->assertStatus(200)
            ->assertJsonStructure([
                'data' => [
                    '*' => ['id', 'title', 'description', 'image', 'author'],
                ],
            ]);
    }

    public function test_can_create_post_when_authenticated(): void
    {
        $response = $this->actingAs($this->user)
            ->postJson('/api/v1/posts', [
                'title' => 'Test Post',
                'description' => 'Test Description',
            ]);

        $response->assertStatus(201)
            ->assertJsonPath('data.title', 'Test Post');

        $this->assertDatabaseHas('posts', [
            'title' => 'Test Post',
        ]);
    }

    public function test_cannot_create_post_when_not_authenticated(): void
    {
        $response = $this->postJson('/api/v1/posts', [
            'title' => 'Test Post',
            'description' => 'Test Description',
        ]);

        $response->assertStatus(401);
    }
}
```

Ejecuta los tests:

```
sail artisan test
```

1.13 Características avanzadas

1.13.1 Rate Limiting

En `routes/api.php`, usa el middleware de throttle:

```
Route::middleware('throttle:60,1')->group(function () {
    Route::post('/auth/login', [AuthController::class, 'login']);
    Route::post('/auth/register', [AuthController::class, 'register']);
});
```

1.13.2 Paginación personalizada

Modifica la respuesta en el controlador:

```
public function index(): Response
{
    $perPage = request()->query('per_page', 15);
    $posts = Post::where('hidden', false)
        ->latest()
        ->paginate($perPage);

    return response([
        'data' => PostResource::collection($posts),
        'pagination' => [
            'total' => $posts->total(),
            'per_page' => $posts->perPage(),
            'current_page' => $posts->currentPage(),
            'last_page' => $posts->lastPage(),
            'from' => $posts->firstItem(),
            'to' => $posts->lastItem(),
        ],
    ]);
}
```

1.13.3 Búsqueda de posts

Añade un scope al modelo Post:

```
<?php

namespace App\Models;

class Post extends Model
{
    public function scopeSearch($query, $term)
    {
        return $query->where('title', 'like', "%{$term}%")
            ->orWhere('description', 'like', "%{$term}%");
    }
}
```

En el controlador:

```
public function index(): Response
{
    $search = request()->query('search');
    $query = Post::where('hidden', false);
```

```
    if ($search) {
        $query->search($search);
    }

    $posts = $query->latest()->paginate(15);

    return response([
        'data' => PostResource::collection($posts),
    ]);
}
```

1.13.4 Autenticación con verificación de email

En el controlador de autenticación:

```
public function register(Request $request): Response
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|string|email|max:255|unique:users',
        'password' => ['required', 'confirmed', Password::defaults()],
    ]);

    $user = User::create([
        'name' => $validated['name'],
        'email' => $validated['email'],
        'password' => Hash::make($validated['password']),
    ]);

    // Enviar notificación de verificación
    $user->sendEmailVerificationNotification();

    $token = $user->createToken('auth_token')->plainTextToken;

    return response([
        'user' => $user,
        'token' => $token,
        'message' => 'Check your email to verify your account',
    ], Response::HTTP_CREATED);
}
```

1.14 Diferencias clave: Laravel 10 vs Laravel 11+

Aspecto	Laravel 10	Laravel 11+
Autenticación preferida	JWT o Sanctum	Sanctum
Estructura de rutas	Grupo en api.php	Grupos más flexibles
Middleware	En Kernel.php	En bootstrapping mejorado
Validación	Request classes	Request classes (sin cambios)
Storage	public_path()	Storage con discos
Tokens	Manual o librería	Integrado en Sanctum
CORS	Manual	config/cors.php
Excepciones	Handler.php	Handler mejorado
Testing	PHPUnit	PHPUnit + Pest

1.15 Deployment

1.15.1 Variables de entorno para producción

Copia el archivo `.env.example` a `.env`:

```
cp .env.example .env
```

Configura:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://api.tudominio.com

DB_CONNECTION=mysql
DB_HOST=db.tuhosting.com
DB_PORT=3306
DB_DATABASE=api_db
DB_USERNAME=user
DB_PASSWORD=password

SANCTUM_STATEFUL_DOMAINS=app.tudominio.com
SESSION_DOMAIN=.tudominio.com
```

1.15.2 Comandos de deployment

```
# Generar clave de aplicación
php artisan key:generate

# Generar clave de JWT (solo si usas JWT adicional)
# php artisan jwt:secret

# Ejecutar migraciones
php artisan migrate --force

# Optimizar la aplicación
php artisan optimize

# Limpiar caché
php artisan cache:clear
```

```
php artisan config:cache
```

1.16 Conclusión

Laravel 11+ proporciona un framework mucho más robusto y seguro para crear APIs REST. Con Sanctum como autenticación preferida, mejor manejo de excepciones, y una estructura más limpia, es más fácil que nunca construir APIs profesionales y escalables.

1.17 Referencias y recursos

- Laravel Sanctum Documentation
- API Resources
- Eloquent API Resources
- Testing
- Validation